

처음 제시하여 받은 코드를 테스트 해보았을 때 고급 과제에서 제시하는 기능을 모두 만족하는 것을 확인하였으나, 코드가 중복되거나 여러 방식의 구조를 섞는 면에서 리팩토링이 필요하다고 생각하였다. 따라서 이를 중점으로 프롬프트를 작성하여 수정된 코드를 받았다. 아래는 도서 대출 관리 시스템 코드다.

1. 코드 품질 관점에서 개선이 필요한 부분을 분석해줘.

(중복 코드, 구조적 일관성, 예외 처리 등을 중심으로)

2. 분석 결과 중 가장 중요한 개선 과제를 선정하여 코드를 리팩터링해줘.

단, 이 작업은 코드 품질 개선 보고서 작성을 전제로 하므로,

개선 근거를 명확히 설명할 수 있는 구조로 수정해줘.

3. 각 개선 사항에 대해 다음 형식으로 정리해줘:

- 기존 코드와 수정 코드를 코드 블록으로 나란히 제시

- "기존 코드에서는 ~한 문제가 있었기 때문에, 다음과 같이 수정하였다.

해당 수정은 ~를 통해 ~를 개선하였다." 형식의 설명을 코드 블록 아래에 첨부

4. 설명은 간결하게 작성하되, 문제점-수정-개선 효과의 논리적 흐름이

명확히 드러나도록 해줘.

3. 오류 및 개선점 분석

테스트 결과 대출/반납/상태 조회 등 요구 기능은 모두 정상 동작하였으나, 코드 품질 측면에서 다음과 같은 문제점이 발견되었다.

인자 개수 검증 로직 중복: `handle_add`, `handle_borrow`, `handle_return`, `handle_status`, `handle_user` 함수마다 `if len(args) != N: print(ERROR...); return` 형태의 동일한 검증 코드가 반복되고 있었다. 검증 방식을 바꾸려면 모든 함수를 일일이 수정해야 하는 유지보수 부담이 있었다.

명령어 분기 구조의 일관성 부족: `main()` 내부에서 `if-elif` 체인으로 명령어를 분기하고 있었으며, 특히 `list` 명령어만 다른 함수들과 달리 `main()`에서 별도로 인자를 검증하고 있어 함수 시그니처와 처리 방식이 통일되어 있지 않았다.

예외 처리 범위의 제한: `input()` 단계에서 발생하는 `KeyboardInterrupt`, `EOFError`만 처리하고 있어, 명령어 처리 함수 내부에서 예상치 못한 오류가 발생하면 프로그램 전체가 종료될 위험이 있었다.

이 중 코드 품질 개선 관점에서 가장 중요한 과제로 중복 코드 제거와 구조 일관성을 선정하였고, 안정성 측면에서 예외 처리 확장도 함께 반영하였다.